

easy-db Java SDK 开发指导书

一、模块概述

1.1 什么是 SDK?

SDK (Software Development Kit, 软件开发工具包) 是一组库、工具和文档的集合, 让其他开发者可以方便地集成和使用你的服务。

本次任务: 为你的 `easy-db` 数据库提供一个 **Java 语言 SDK**, 让其他 Java 开发者可以通过简单的 API 调用操作数据库, 而无需关心底层 Socket 通信细节。

1.2 为什么需要 SDK?

用途	说明
降低使用门槛	用户只需 <code>client.set("key", "value")</code> , 无需了解 Socket 通信协议
提升开发效率	提供类型安全的 API, IDE 自动补全, 减少拼写错误
封装复杂性	将连接管理、重试、序列化等复杂逻辑封装在 SDK 内部
推广服务	让更多开发者能够快速接入你的数据库服务

1.3 效果预览

```
// 用户只需要这几行代码
DatabaseClient client = new DatabaseClient("localhost", 8080);
client.connect();

client.set("name", "张三");
String name = client.get("name");
System.out.println(name); // 输出: 张三

client.close();
```

二、功能需求

2.1 核心功能 (必做)

需求 2.1.1: 连接管理

封装与 `easy-db` 服务端的连接管理。

```
public class DatabaseClient {
    private String host;
    private int port;
    private Socket socket;
    private PrintWriter out;
    private BufferedReader in;
    private boolean connected;

    // 构造方法
    public DatabaseClient(String host, int port) {
```

```

        this.host = host;
        this.port = port;
    }

    // 连接服务端
    public void connect() throws IOException {
        socket = new Socket(host, port);
        out = new PrintWriter(socket.getOutputStream(), true);
        in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        connected = true;
    }

    // 断开连接
    public void close() {
        // 关闭资源
    }
}

```

验收标准：

- ☐ `connect()` 能成功连接服务端
- ☐ `close()` 正确释放资源
- ☐ 连接失败时抛出友好异常

需求 2.1.2：核心命令 API

对应服务端的命令，提供类型安全的 Java 方法。

```

/**
 * 存储键值对
 */
public String set(String key, String value) {
    return sendCommand("SET " + key + " " + value);
}

/**
 * 获取键对应的值
 */
public String get(String key) {
    return sendCommand("GET " + key);
}

/**
 * 删除键值对
 */
public String del(String key) {
    return sendCommand("DEL " + key);
}

/**
 * 列出所有键
 */
public List<String> keys(String pattern) {
    String result = sendCommand("KEYS " + pattern);
    return parseListResult(result);
}

```

```

/**
 * 检查键是否存在
 */
public boolean exists(String key) {
    String result = sendCommand("EXISTS " + key);
    return "true".equals(result);
}

/**
 * 清空所有数据
 */
public String flush() {
    return sendCommand("FLUSH");
}

```

验收标准:

- ☐ 每个 API 方法能正确调用服务端对应命令
- ☐ 返回值类型正确 (String、List、boolean)
- ☐ 错误时抛出异常

需求 2.1.3: 命令发送与响应解析

封装 Socket 通信的核心逻辑。

```

private String sendCommand(String command) {
    if (!connected) {
        throw new IllegalStateException("Not connected to server");
    }

    try {
        // 发送命令
        out.println(command);

        // 读取响应
        StringBuilder response = new StringBuilder();
        String line;
        while ((line = in.readLine()) != null) {
            // 如果响应是单行, 直接返回
            // 如果是多行 (如 KEYS 命令), 拼接后返回
            response.append(line);
            // 简单判断: 如果响应以 ( 开头, 可能是多行结果
            // 更精确的协议解析见下方进阶需求
        }
        return response.toString();
    } catch (IOException e) {
        throw new RuntimeException("Failed to execute command: " + command, e);
    }
}

private List<String> parseListResult(String result) {
    // 解析 KEYS 命令返回的多行结果
    // 格式:
    // 1) "key1"
    // 2) "key2"
    List<String> list = new ArrayList<>();
    if (result == null || result.isEmpty()) return list;
}

```

```
String[] lines = result.split("\n");
for (String line : lines) {
    String trimmed = line.trim();
    if (trimmed.matches("^\\d+\\)\\s+\"(.*)\"$")) {
        // 提取引号中的内容
        String key = trimmed.replaceAll("^\\d+\\)\\s+\"|\"$", "");
        list.add(key);
    }
}
return list;
}
```

2.2 增强功能（选做加分）

需求 2.2.1：连接池支持

支持复用连接，减少频繁创建 Socket 的开销。

```
public class ConnectionPool {
    private List<DatabaseClient> clients;
    private int maxSize = 10;

    public DatabaseClient borrowClient() {
        // 从池中获取一个可用连接
    }

    public void returnClient(DatabaseClient client) {
        // 归还连接
    }
}
```

需求 2.2.2：自动重连

网络异常时自动尝试重连。

```
private String sendCommandWithRetry(String command, int maxRetries) {
    int attempt = 0;
    while (attempt < maxRetries) {
        try {
            return sendCommand(command);
        } catch (IOException e) {
            attempt++;
            if (attempt < maxRetries) {
                // 等待后重试
                Thread.sleep(1000 * attempt);
                reconnect();
            } else {
                throw new RuntimeException("Failed after " + maxRetries + " attempts", e);
            }
        }
    }
}
```

需求 2.2.3：异步调用

使用 `CompletableFuture` 支持非阻塞调用。

```
public CompletableFuture<String> setAsync(String key, String value) {  
    return CompletableFuture.supplyAsync(() -> set(key, value));  
}
```

需求 2.2.4：连接状态监听

提供连接状态变化的事件回调。

```
public interface ConnectionListener {  
    void onConnected();  
    void onDisconnected();  
    void onError(Exception e);  
}
```

三、代码设计

3.1 包结构

在 `client` 包下新建 `sdk` 子包：

```
src/main/java/client/  
├─ Client.java           # 已有  
├─ SocketClient.java     # 已有  
├─ sdk/  
│   ├── DatabaseClient.java # 核心客户端类  
│   ├── ConnectionPool.java # 连接池（选做）  
│   └─ exception/  
│       └─ DatabaseException.java  
├─ cli/                  # 已有  
└─ orm/                  # 已有
```

3.2 核心类实现

DatabaseClient.java

```
package client.sdk;  
  
import java.io.*;  
import java.net.Socket;  
import java.util.*;  
  
/**  
 * easy-db Java SDK 客户端  
 *  
 * 用法：  
 * <pre>  
 * DatabaseClient client = new DatabaseClient("localhost", 8080);  
 * client.connect();  
 * client.set("name", "张三");  
 * String name = client.get("name");  
 * client.close();  
 * </pre>  
 */  
public class DatabaseClient {
```

```

private final String host;
private final int port;
private final int timeout;

private Socket socket;
private PrintWriter out;
private BufferedReader in;
private boolean connected;

// ===== 构造方法 =====

public DatabaseClient(String host, int port) {
    this(host, port, 5000);
}

public DatabaseClient(String host, int port, int timeout) {
    this.host = host;
    this.port = port;
    this.timeout = timeout;
}

// ===== 连接管理 =====

/**
 * 连接到服务端
 */
public void connect() throws DatabaseException {
    try {
        socket = new Socket();
        socket.connect(new InetSocketAddress(host, port), timeout);
        socket.setSoTimeout(timeout);

        out = new PrintWriter(socket.getOutputStream(), true);
        in = new BufferedReader(new InputStreamReader(socket.getInputStream(),
StandardCharsets.UTF_8));

        connected = true;
    } catch (IOException e) {
        throw new DatabaseException("Failed to connect to " + host + ":" + port, e);
    }
}

/**
 * 检查是否已连接
 */
public boolean isConnected() {
    return connected && socket != null && !socket.isClosed();
}

/**
 * 关闭连接
 */
public void close() {
    try {
        if (in != null) in.close();
        if (out != null) out.close();
        if (socket != null) socket.close();
    } catch (IOException e) {

```

```

        // 忽略关闭异常
    } finally {
        connected = false;
    }
}

// ===== 核心命令 API =====

/**
 * 存储键值对
 */
public String set(String key, String value) throws DatabaseException {
    checkConnected();
    return sendCommand("SET " + key + " " + value);
}

/**
 * 获取键对应的值
 */
public String get(String key) throws DatabaseException {
    checkConnected();
    return sendCommand("GET " + key);
}

/**
 * 删除键值对
 */
public String del(String key) throws DatabaseException {
    checkConnected();
    return sendCommand("DEL " + key);
}

/**
 * 列出所有键（支持通配符）
 */
public List<String> keys(String pattern) throws DatabaseException {
    checkConnected();
    String result = sendCommand("KEYS " + (pattern == null ? "*" : pattern));
    return parseMultiLineResponse(result);
}

/**
 * 检查键是否存在
 */
public boolean exists(String key) throws DatabaseException {
    checkConnected();
    String result = sendCommand("EXISTS " + key);
    return "true".equalsIgnoreCase(result.trim());
}

/**
 * 清空所有数据
 */
public String flush() throws DatabaseException {
    checkConnected();
    return sendCommand("FLUSH");
}

```

```
// ===== 命令执行核心 =====

private String sendCommand(String command) throws DatabaseException {
    try {
        out.println(command);

        // 读取第一行, 判断响应类型
        String firstLine = in.readLine();
        if (firstLine == null) {
            throw new DatabaseException("Server returned null response");
        }

        // 如果是多行响应 (如 KEYS 命令), 继续读取
        if (firstLine.isEmpty()) {
            // 空响应
            return "";
        }

        // 检查是否是多行响应: 以 "1)" 开头表示列表
        if (firstLine.trim().matches("^\\d+\\s")) {
            StringBuilder sb = new StringBuilder(firstLine);
            String line;
            while ((line = in.readLine()) != null) {
                if (line.trim().isEmpty()) break;
                sb.append("\\n").append(line);
            }
            return sb.toString();
        }

        // 单行响应
        return firstLine;
    } catch (IOException e) {
        connected = false;
        throw new DatabaseException("Failed to execute command: " + command, e);
    }
}

private List<String> parseMultiLineResponse(String response) {
    List<String> result = new ArrayList<>();
    if (response == null || response.isEmpty()) {
        return result;
    }

    String[] lines = response.split("\\n");
    for (String line : lines) {
        String trimmed = line.trim();
        // 匹配格式: 1) "value"
        java.util.regex.Pattern pattern = java.util.regex.Pattern.compile("^\\d+\\s\\s+\\s+\"(.*)\"\\s$");
        java.util.regex.Matcher matcher = pattern.matcher(trimmed);
        if (matcher.find()) {
            result.add(matcher.group(1));
        }
    }
    return result;
}

```



```

private void checkConnected() {
    if (!isConnected()) {
        throw new IllegalStateException("Not connected to server. Call connect() first.");
    }
}
}
}

```

DatabaseException.java

```

package client.sdk.exception;

public class DatabaseException extends RuntimeException {

    public DatabaseException(String message) {
        super(message);
    }

    public DatabaseException(String message, Throwable cause) {
        super(message, cause);
    }
}

```

四、使用示例

4.1 基础使用

```

package client.sdk.example;

import client.sdk.DatabaseClient;
import client.sdk.exception.DatabaseException;

public class SDKExample {

    public static void main(String[] args) {
        DatabaseClient client = new DatabaseClient("localhost", 8080);

        try {
            // 1. 连接服务端
            client.connect();
            System.out.println("✅ 连接成功!");

            // 2. 存储数据
            client.set("name", "张三");
            System.out.println("✅ 存储成功");

            client.set("age", "25");
            client.set("city", "广州");

            // 3. 查询数据
            String name = client.get("name");
            System.out.println("姓名: " + name);

            // 4. 列出所有键
            List<String> keys = client.keys("*");
            System.out.println("所有键: " + keys);
        } catch (DatabaseException e) {
            e.printStackTrace();
        }
    }
}

```

```

// 5. 检查存在
boolean exists = client.exists("name");
System.out.println("name 存在: " + exists);

// 6. 删除数据
client.del("age");
System.out.println("✅ age 已删除");

// 7. 再次查询
String age = client.get("age");
System.out.println("age: " + (age == null ? "(nil)" : age));

} catch (DatabaseException e) {
    System.err.println("❌ 错误: " + e.getMessage());
    e.printStackTrace();
} finally {
    // 8. 关闭连接
    client.close();
    System.out.println("✅ 连接已关闭");
}
}
}

```

运行结果:

```

✅ 连接成功!
✅ 存储成功
姓名: 张三
所有键: [name, city, age]
name 存在: true
✅ age 已删除
age: (nil)
✅ 连接已关闭

```

4.2 与 ORM 模块配合使用

将 SDK 与 ORM 模块集成, 实现对象存储:

```

// SDK 提供底层通信, ORM 提供对象映射
DatabaseClient client = new DatabaseClient("localhost", 8080);
client.connect();

// 配合 ORM 使用
User user = new User("001", "张三", 20);
String json = userToJson(user); // 序列化
client.set("user:001", json);   // 存储

String result = client.get("user:001");
User cached = jsonToUser(result); // 反序列化

```

4.3 批量操作示例

```

public class BatchExample {
    public static void main(String[] args) {
        DatabaseClient client = new DatabaseClient("localhost", 8080);
        client.connect();
    }
}

```

```
// 批量插入
Map<String, String> data = new HashMap<>();
data.put("user:001", "张三");
data.put("user:002", "李四");
data.put("user:003", "王五");
data.put("product:001", "手机");
data.put("product:002", "电脑");

for (Map.Entry<String, String> entry : data.entrySet()) {
    client.set(entry.getKey(), entry.getValue());
}
System.out.println("✅ 批量插入完成");

// 批量查询
List<String> keys = client.keys("user:*");
for (String key : keys) {
    String value = client.get(key);
    System.out.println(key + " = " + value);
}

client.close();
}
```

五、开发任务清单

阶段	任务	状态
阶段一	创建 sdk 包和 DatabaseClient 类骨架	☐
阶段二	实现 connect() 和 close() 连接管理	☐
阶段三	实现 sendCommand() 核心通信方法	☐
阶段四	实现 set()、get()、del() 三个核心命令	☐
阶段五	实现 keys()、exists()、flush() 命令	☐
阶段六	实现多行响应解析 (parseMultiLineResponse)	☐
阶段七	编写单元测试或示例代码验证	☐
阶段八 (选做)	连接池实现	☐
阶段九 (选做)	自动重连机制	☐

六、常见问题

Q1: SDK 和 CLI 工具、Shell 工具有什么区别?

工具	用途	使用方式
CLI 工具	人工交互式操作	<code>easy-db> SET name 张三</code>
Shell 工具	脚本调用、管道组合	<code>easy-db set name 张三</code>
Java SDK	Java 程序集成	<code>client.set("name", "张三")</code>

Q2: `sendCommand()` 中如何判断响应是否是多行?

A: 可以根据响应格式判断。`KEYS` 命令返回多行, 格式为 `1) "value"`, 第一行以数字开头。可以先读第一行判断, 再决定是否继续读取。

Q3: 如何处理中文乱码?

A: 确保使用 UTF-8 编码:

```
new BufferedReader(new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8))
```

Q4: 如果服务端没有启动, SDK 应该怎么办?

A: `connect()` 时应抛出 `DatabaseException`, 并包含清晰的错误信息:

```
Failed to connect to localhost:8080
Caused by: java.net.ConnectException: Connection refused
```

Q5: `keys("*")` 返回的数据量大怎么办?

A: 可以在 SDK 中提供分页支持, 或增加 `SCAN` 命令 (选做)。

七、提交要求

1. 代码: SDK 模块全部源码, Git 提交记录
2. 示例: 提供 `SDKExample.java` 演示所有功能
3. 报告: 在课程设计报告中增加 **Java SDK** 章节, 内容包括:
 - API 设计思路
 - 核心类说明
 - 使用示例截图 (至少 3 张)
 - 遇到的困难与解决
4. 演示: 验收时现场演示 SDK 的使用

八、参考资料

- [Java SDK 设计最佳实践](#)
- [Java Socket 编程](#)
- [Java 异常处理](#)

祝同学们顺利完成 Java SDK 开发!